

Adaptive Noise Cancellation of Speech Using LMS and NLMS Algorithms

Md Rumman Huda Sarkar

Student ID: 40293223

ELEC 6651 – Adaptive Signal Processing

Contents

1	Introduction	2
2	Project Work and Methods	3
2.1	Signal Model and Adaptive Noise Cancellation Structure	3
2.2	LMS Algorithm	3
2.3	NLMS Algorithm	4
2.4	Methodology	5
2.5	Evaluation Metrics	6
3	Simulation and Numerical Results	7
3.1	Time-Domain Waveform Analysis	7
3.2	Convergence Behavior Analysis	7
3.3	Spectrogram Analysis	8
3.4	SNR-Based Benchmarking Results	10
3.5	Parameter Sensitivity Analysis	12
4	Limitations	12
5	Future Work	12
6	Conclusion	13
	References	14
A	Python Code for LMS and NLMS Adaptive Noise Cancellation	15

Abstract

The speech data gathered under real-life conditions will often contain additive noise that adversely affects the quality and clarity of speech. One solution to this problem involves applying the concept of adaptive noise cancellation. In order to apply adaptive noise cancellation effectively, there must be a reference noise signal that correlates with the unwanted noise. This work examines the application of the Least Mean Squares (LMS) algorithm and Normalized Least Mean Squares (NLMS) algorithm for adaptive speech enhancement. Speech signal in this case is viewed as the summation of a clean speech signal plus some noise. The reference noise signal is fed into an adaptive finite impulse response filter that models noise and the output is subtracted from the speech signal. Performance of LMS and NLMS algorithm is measured using SNR improvement, mean-square error, waveform comparison, rate of convergence, spectrogram analysis and audio playback. From the results obtained, NLMS outperformed LMS in terms of noise suppression and output SNR. The maximum LMS output SNR value was 15.05 dB compared to 17.58 dB for NLMS. The findings also reveal that decorrelation delay exerts the most significant effect on performance, while step size follows, whereas filter length exerts relatively less effect. In general, the NLMS algorithm exhibits superior convergence characteristics, enhanced stability, and speech enhancement performance.

Keywords— Adaptive noise cancellation, LMS, NLMS, speech enhancement, adaptive filtering, signal-to-noise ratio, mean-square error.

1 Introduction

Speech communication systems are commonly faced with background noises, like those from traffic, machinery, crowd noises, air conditioning, or other noises. Such noises could impair the quality of speech signals, leading to difficulties in applications, including mobile communication systems, hearing aids, voice-operated devices, video and audio conferencing, and speech recognition.

Fixed filters may not be a viable option for speech enhancement since real-life noises are typically dynamic and cannot be determined beforehand. An alternative approach that could offer adaptability is the use of adaptive filters, where filter weights are automatically adjusted in response to changes in input signals and errors. Within adaptive filters, the Least Mean Squares (LMS) is a common method due to simplicity and ease of computation. Nevertheless, LMS depends critically on the choice of step size and input power. A modification to LMS involves normalized Least Mean Squares, which normalizes coefficient update in relation to the signal energy input.

The purpose of the project will be the design and assessment of adaptive noise cancellation methods for speech signal enhancement using adaptive filters based on the LMS algorithm. The comparison between LMS and NLMS will be done using noisy speech samples. The assessment will include time domain analysis of the waveform, spectrogram analysis, convergence analysis, SNR analysis, and mean square error analysis.

2 Project Work and Methods

2.1 Signal Model and Adaptive Noise Cancellation Structure

In adaptive noise cancellation, the observed noisy speech signal is modeled as

$$x[n] = s[n] + v[n], \quad (1)$$

where $x[n]$ is the observed noisy signal, $s[n]$ is the clean speech signal, and $v[n]$ is the additive noise component.

A reference noise signal $v_r[n]$ is assumed to be available. This reference signal should be highly correlated with the actual noise $v[n]$, but uncorrelated with the clean speech signal $s[n]$. The purpose of the adaptive filter is to process $v_r[n]$ and generate an estimate of the noise, denoted as $\hat{v}[n]$. This estimated noise is then subtracted from the noisy signal:

$$e[n] = x[n] - \hat{v}[n], \quad (2)$$

where $e[n]$ stands for the error signal. In this case, the error signal is also used as the output speech signal. The adaptive filter keeps changing its coefficients so that the noise estimate approaches the real noise present in the noisy speech signal.

The adaptive noise cancellation system has the following components: the primary input, the reference input, the adaptive FIR filter, the subtractor, and the coefficient updater. The primary input consists of speech and noise, whereas the reference input comprises a noise signal. The output of the adaptive FIR filter is the noise estimate, whereas the subtraction yields the enhanced speech signal.

2.2 LMS Algorithm

The least mean squares algorithm is a stochastic gradient adaptive filter algorithm. It uses the instantaneous error signal and the reference input vector in order to update the filter coefficients. The LMS algorithm tries to minimize the mean-square error between the desired

signal and the adaptive filter output.

Let the adaptive filter weight vector at time n be

$$\mathbf{w}[n] = [w_0[n], w_1[n], \dots, w_{L-1}[n]]^T, \quad (3)$$

and the reference input vector be

$$\mathbf{v}_r[n] = [v_r[n], v_r[n-1], \dots, v_r[n-L+1]]^T, \quad (4)$$

where L is the FIR filter length. The adaptive filter output is

$$\hat{v}[n] = \mathbf{w}^T[n] \mathbf{v}_r[n]. \quad (5)$$

The enhanced output or error signal is

$$e[n] = x[n] - \hat{v}[n]. \quad (6)$$

The LMS weight update equation is

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n] \mathbf{v}_r[n], \quad (7)$$

with μ being the step-size. The step size regulates the rate of adaptation of the filter coefficients. If the step size μ is too small, then there will be a slow adaptation, whereas, if μ is large, there is likely to be instability or divergence of the algorithm. The LMS is easy to compute and can be implemented in real time.

2.3 NLMS Algorithm

Normalization of the Least Mean Squares algorithm involves modifying the update formula for the LMS algorithm through normalization of the step size with respect to the energy of the reference input vector. This is aimed at preventing instability caused by variations in the input signal.

The NLMS update equation is

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \frac{\mu}{\|\mathbf{v}_r[n]\|^2 + \epsilon} e[n] \mathbf{v}_r[n], \quad (8)$$

with ϵ being a very small positive number employed for avoiding division by zero. NLMS adjusts its step size relative to the power of the reference signal. For that reason, the NLMS

algorithm is more suitable for speech signals that are inherently non-stationary and have time-varying amplitude values.

NLMS has an advantage over LMS in terms of increased stability. The disadvantage of NLMS is its slightly greater computation complexity due to the necessity of computing the energy of the input vector. In the current research, it is assumed that NLMS will demonstrate better performance than LMS due to the non-stationary nature of speech signals.

2.4 Methodology

This experimentation was carried out in Python programming language. Clean speech and noisy speech signals were imported from audio files, and normalization was done to prevent problems caused by amplitude scaling. In case stereo audio was identified, conversion to mono audio was done. Clean and noisy speech signals were then truncated to have equal length.

The true noise signal was obtained as

$$v[n] = x[n] - s[n]. \quad (9)$$

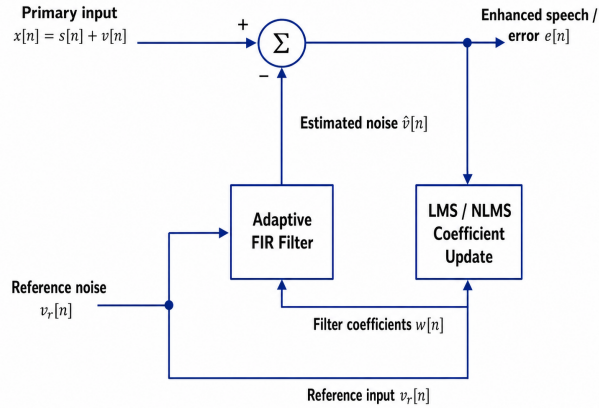


Figure 1: Block diagram of the LMS/NLMS-based adaptive noise cancellation system.

For the purpose of experiment, the reference noise signal was created from the original noise signal. Another function called delay was also used to create different correlations between the reference noise signal and the original noise signal. Three different delays were used in the process: 0, 5, and 10. An increased delay reduces the correlation between the reference noise and original noise, thus making ANF difficult.

The LMS and NLMS techniques were employed for filtering the speech signal under

different parameter settings. The step sizes considered were

$$\mu = 0.001, 0.005, 0.01. \quad (10)$$

The tested FIR filter lengths were

$$L = 32, 64, 128. \quad (11)$$

The results for each step size, filter length, and delay were generated for both LMS and NLMS algorithms. The results were analyzed based on the output signal-to-noise ratio (SNR), SNR improvement, and mean squared error (MSE). The best LMS and NLMS results were selected based on the maximum output SNR values. The project also provided waveforms, convergence graphs, spectrograms, and enhanced audio clips for analysis.

2.5 Evaluation Metrics

The first evaluation metric used in this project is signal-to-noise ratio. The SNR between the clean signal and a tested signal is calculated as

$$SNR = 10 \log_{10} \left(\frac{\sum s^2[n]}{\sum (s[n] - \hat{s}[n])^2} \right), \quad (12)$$

where $s[n]$ is the clean speech signal and $\hat{s}[n]$ is the noisy or enhanced speech signal. A higher SNR indicates that the enhanced speech is closer to the clean speech.

The SNR improvement is calculated as

$$SNR_{\text{improvement}} = SNR_{\text{output}} - SNR_{\text{input}}. \quad (13)$$

A positive SNR improvement means that the adaptive filter improved the speech signal compared with the original noisy input.

The second metric is mean-square error:

$$MSE = \frac{1}{N} \sum_{n=1}^N (s[n] - \hat{s}[n])^2. \quad (14)$$

The smaller MSE is an indication of improved reconstructions of the clean speech. In analyzing the convergence, the error in noise estimation was determined by comparison of the actual noise and the noise estimated using LMS and NLMS techniques. The results were plotted in dB form for the purposes of viewing learning behavior.

3 Simulation and Numerical Results

3.1 Time-Domain Waveform Analysis

Waveform representation is made for the speech signal, noisy signal, enhanced signal using LMS, and enhanced signal using NLMS. In the waveform of the noisy speech signal, noise is seen to add fluctuations to the speech. Following the process of adaptive filter implementation, there are improvements shown on the performance of noise reduction in LMS and NLMS output waveforms. Nevertheless, the LMS output looks more like the noisy signal, implying poor improvement in noise reduction.

Fig. 2 shows the time-domain waveforms of the clean speech, noisy speech, LMS-enhanced speech, and NLMS-enhanced speech. The noisy speech signal contains additional background fluctuations compared with the clean speech signal. After adaptive filtering, both LMS and NLMS preserve the main speech structure while reducing part of the noise component. The NLMS-enhanced waveform appears closer to the clean speech waveform, especially in the low-amplitude regions between speech segments.

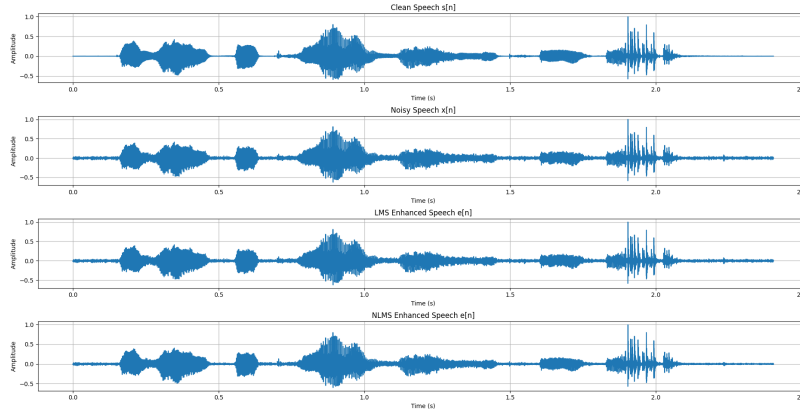


Figure 2: Time-domain waveform comparison of clean speech, noisy speech, LMS-enhanced speech, and NLMS-enhanced speech.

3.2 Convergence Behavior Analysis

The learning curve was obtained from the smoothed noise estimation error in dB. From the graph above, we can see that both LMS and NLMS algorithms estimate the noise at different times. As the error level decreases, the accuracy of estimating the noise increases. From the Fig 3, the NLMS line falls below the LMS line for most of the duration of the signal. Therefore, NLMS is capable of estimating the noise more efficiently than LMS, especially at

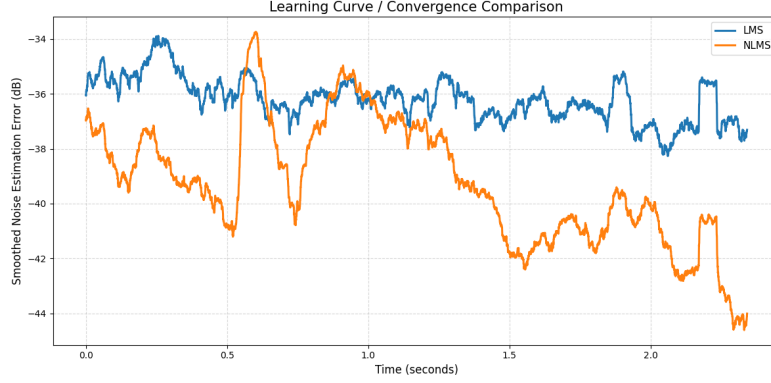


Figure 3: Learning curve and convergence comparison between LMS and NLMS algorithms.

the end of the signal where the error levels of the two are compared. This means that NLMS is able to converge faster than LMS.

3.3 Spectrogram Analysis

Analysis of spectrograms was made in order to compare the difference in the frequency-time characteristics of clean, noisy, enhanced by LMS and NLMS speech signals. The spectrogram of the clean speech signal contains more clearly defined speech areas with visible periods of silence between different speech fragments. The spectrogram of the noisy speech signal contains more background noise energy in the low and medium frequency ranges.

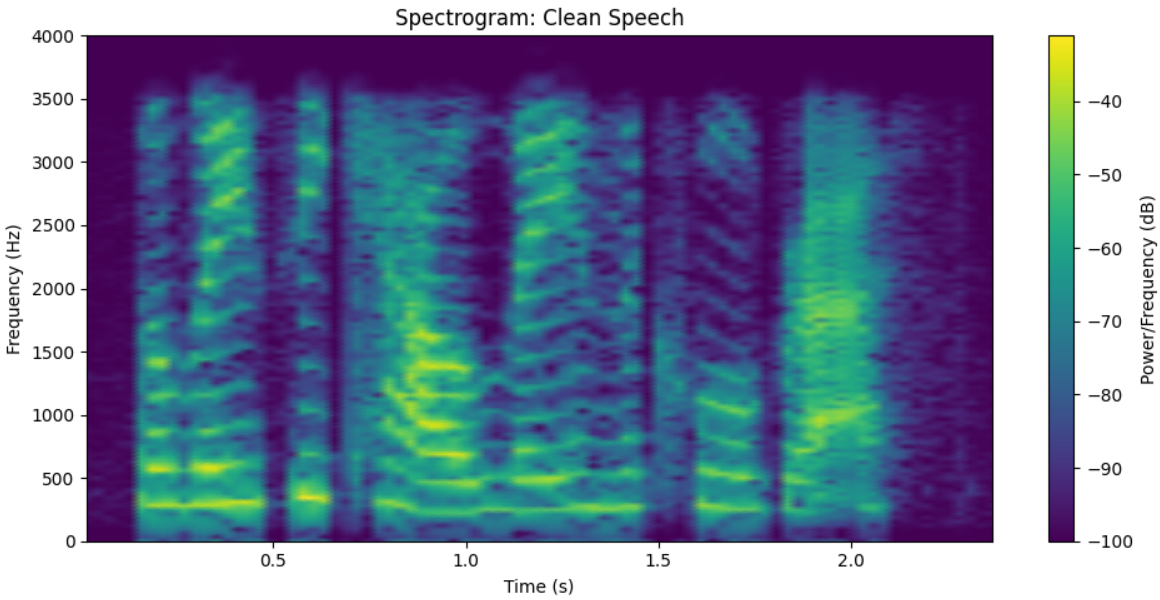


Figure 4: Spectrogram of the clean speech signal.

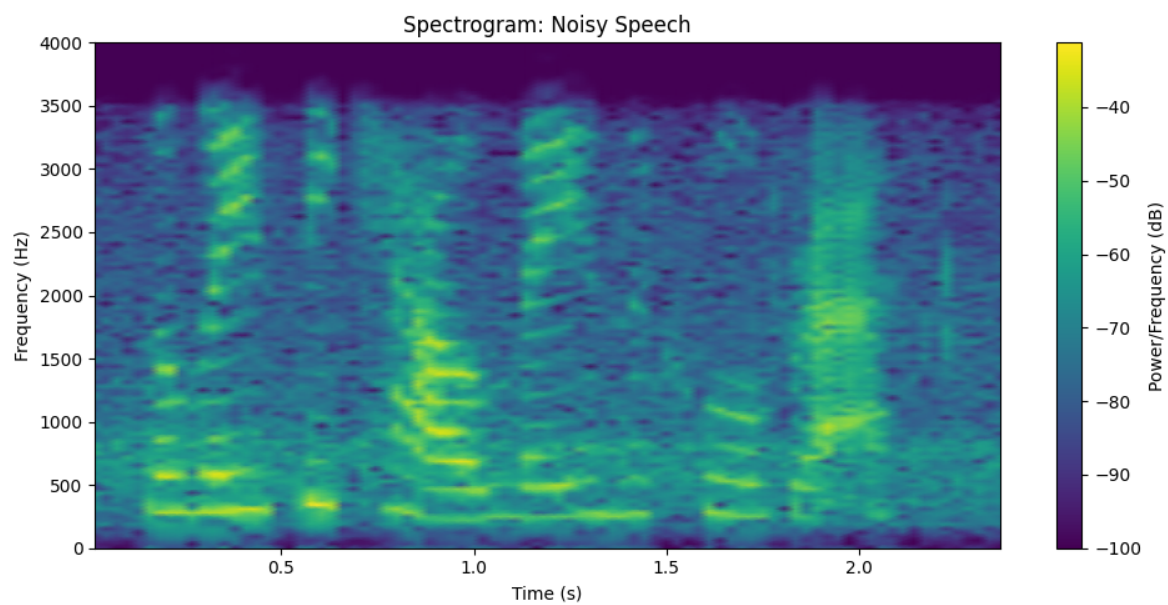


Figure 5: Spectrogram of the noisy speech signal.

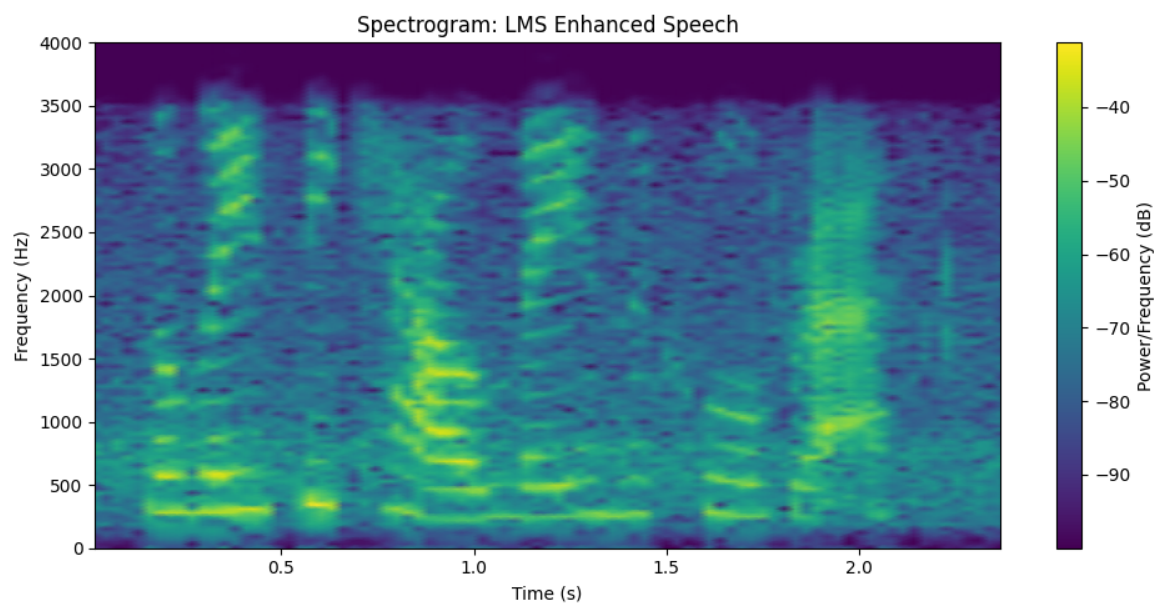


Figure 6: Spectrogram of the LMS-enhanced speech signal.

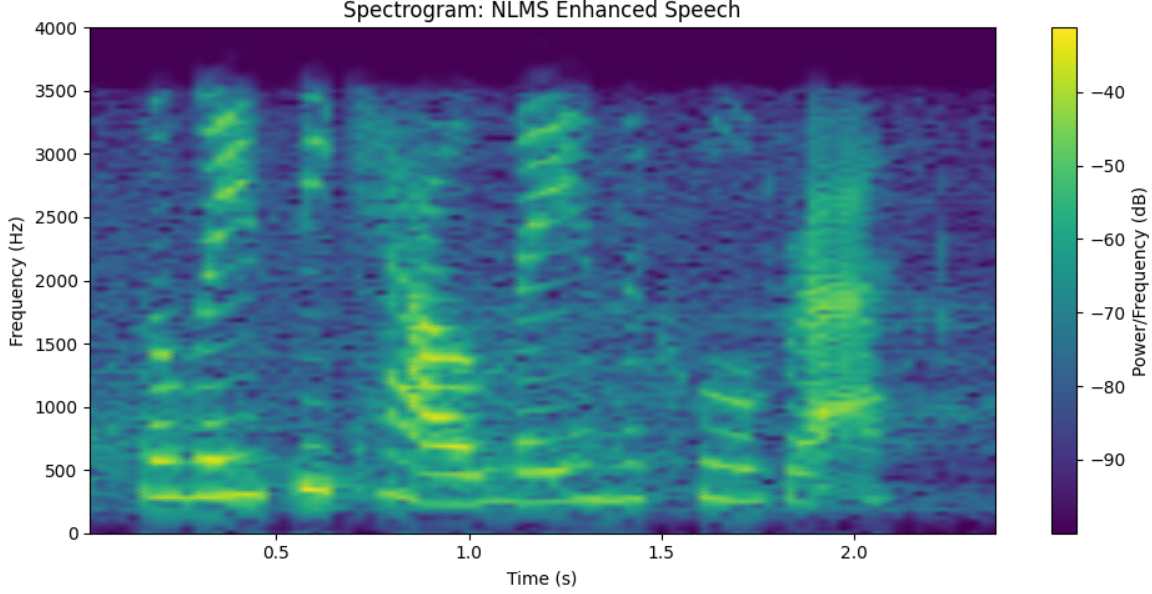


Figure 7: Spectrogram of the NLMS-enhanced speech signal.

The spectrogram of enhanced by LMS speech signal looks very similar to the spectrogram of the noisy speech signal which means that LMS algorithm does suppress part of the noise but does not reduce the background noise interference significantly. The spectrogram of enhanced by NLMS speech signal contains less background noise in comparison to the two previous spectrograms especially in the background noise zones between speech fragments. Moreover, speech formants and harmonics were not destroyed during both LMS and NLMS enhancement.

As seen from the spectrograms, NLMS enhances the signal without destroying the speech structure much better than LMS.

3.4 SNR-Based Benchmarking Results

Table 1 summarizes selected benchmarking results for LMS and NLMS using different filter lengths and delay values.

It is observed that NLMS outperforms LMS in terms of SNR, especially at zero delays. When considering the case where $L = 32$ and there is no delay, the former improves SNR by merely 0.04 dB whereas the latter improves the SNR by 2.54 dB. The fact indicates that NLMS performs much better than LMS for cases in which the reference noise and actual noise are highly correlated.

The performance of both algorithms decreases as the delay changes from 0 to 10. It occurs since, as mentioned previously, the higher delay leads to lower correlation between

Table 1: Selected Benchmarking Results for LMS and NLMS

L	Delay	LMS SNR (dB)	NLMS SNR (dB)	LMS Gain (dB)	NLMS Gain (dB)
32	0	14.66	17.16	0.04	2.54
32	5	14.64	15.46	0.02	0.85
32	10	14.63	14.75	0.01	0.14
64	0	14.67	16.36	0.05	1.74
64	5	14.64	15.23	0.02	0.61
64	10	14.63	14.74	0.01	0.13

reference noise and actual noise, making the adaptive noise cancellation less effective. The reason is that adaptive noise cancellation relies heavily on the correlation between noise signals.

However, when the filter length is increased from 32 to 64, there is not much improvement in terms of LMS performance. In the case of NLMS, the best result in Table 1 is obtained when $L = 32$. It indicates that an increase in the filter length is not a key factor in obtaining better results since a larger filter can lead to more complex computations.

Table 2: Best Performing Parameter Settings for LMS and NLMS

Algorithm	μ	L	Delay	Output SNR (dB)	Improvement (dB)
LMS	0.010	64	0	15.05	0.43
NLMS	0.005	32	0	17.58	2.96

Table 2 summarizes the best performing parameter settings obtained from the full parameter sweep. The best LMS performance was achieved with $\mu = 0.010$, filter length $L = 64$, and zero delay, producing an output SNR of 15.05 dB and an improvement of 0.43 dB. In comparison, the best NLMS performance was achieved with $\mu = 0.005$, filter length $L = 32$, and zero delay, producing an output SNR of 17.58 dB and an improvement of 2.96 dB. This confirms that NLMS provides better speech enhancement performance than LMS under the tested conditions.

The complete parameter sweep was performed over 27 combinations of step size, filter length, and delay. For compact presentation, Table 1 shows selected representative results, while Table 2 summarizes the best LMS and NLMS configurations.

3.5 Parameter Sensitivity Analysis

Three major parameters that were analyzed in this research project include step size, filter length, and delay.

Step size μ regulates the speed of adaptation. If the value is low, the convergence of LMS would be slow, while too high a value will result in system instability. Step size was moderately important in determining the performance of the adaptive systems in the current experiments. The LMS algorithm was more sensitive to step size due to its lack of normalization using input power. NLMS had higher stability since its weight update mechanism adjusted itself according to the energy of the reference signal.

Filter length specifies how many previous reference signal samples are used by the adaptive FIR filter. Higher values imply a more complex noise channel but with higher computational complexity and greater convergence time. Filter length was the least important parameter in the present experiment.

The results showed that delay has a significant effect on performance. At zero delay, there is a high correlation between the reference and the actual noise signal, enabling both methods to perform better in estimating the noise. As delay increases, the correlation between the reference and the actual noise decreases, resulting in poorer noise cancellation by the adaptive filter. This is particularly evident in NLMS, where the algorithm exhibits excellent performance at zero delay and relatively poor improvement at higher delays.

4 Limitations

Though these results confirm the success of ANC, there are some limitations to this project. Firstly, the reference noise was acquired on the basis of the difference between the noisy and clear speech. In a practical system, another channel (microphone or other sensor) is needed to obtain the signal correlated with interference. Secondly, only a few examples of speech with various kinds of noises at several signal-to-noise ratio values were considered during the testing. More speakers, noises, and noise levels should be considered in order to conduct a more accurate study. Finally, the effectiveness of noise reduction was evaluated solely on the basis of objective criteria (SNR and MSE).

5 Future Work

Future research can take this project further in multiple directions. To begin with, the filters can be applied to bigger databases like NOIZEUS and LibriSpeech with different kinds of

noises and varying SNRs. Then, the references for the filters can be obtained from actual microphone signals rather than the artificial reference noises. Third, Recursive Least Squares can be included and contrasted with LMS and NLMS owing to its faster performance despite higher complexity. Next, speech intelligibility and perceptual quality criteria like PESQ and STOI can be introduced to analyze the quality of speech. Lastly, adaptive filtering techniques or machine learning-based speech enhancement approaches can be considered against the current LMS-based filters.

6 Conclusion

This paper has designed an adaptive noise canceler model that improves the speech signals by LMS and NLMS algorithms. Noisy speech signal can be considered as a combination of original speech signal and additive noise signal. Reference noise signal is applied on an adaptive FIR filter to estimate noise signal. The estimated noise signal is then subtracted from the input noisy signal to get an enhanced speech signal.

Experimental results demonstrate that both LMS and NLMS are capable of reducing noise levels, and among those two, NLMS has better results than LMS algorithm. NLMS provided better output SNR ratio, good convergence, and better spectrogram results than LMS. The highest output SNR obtained by LMS algorithm is 15.05 dB and by NLMS algorithm is 17.58 dB. From the experiments, we see that the correlation between reference noise signal plays a very crucial role. Increasing delay caused the worst results, especially in the case of NLMS. Effect of step size is moderate, whereas effect of filter length is negligible.

In general, NLMS works better than LMS in this experiment due to its superior properties.

References

- [1] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1985.
- [2] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2014.
- [3] S. A. Hadei and M. Lotfizad, “A family of adaptive filter algorithms in noise cancellation for speech enhancement,” *arXiv preprint arXiv:1106.0846*, 2011.
- [4] P. C. Loizou, “Speech enhancement based on perceptually motivated Bayesian estimators of the speech magnitude spectrum,” *IEEE Transactions on Speech and Audio Processing*, vol. 13, no. 5, pp. 857–869, 2005.
- [5] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.
- [6] M. S. E. Abadi, S. Kadambe, and D. P. W. Ellis, “Performance analysis of LMS and NLMS adaptive algorithms for speech enhancement in noisy environment,” *International Journal of Innovative Technology and Exploring Engineering*, 2020.

A Python Code for LMS and NLMS Adaptive Noise Cancellation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import spectrogram
import pandas as pd

# =====
# Load and Normalize Audio
# =====
def load_and_normalize(filename):
    fs, data = wavfile.read(filename)

    if data.ndim > 1:
        data = data[:, 0]

    data = data.astype(np.float32)

    max_val = np.max(np.abs(data))
    if max_val > 0:
        data = data / max_val

    return fs, data

# =====
# Save Audio
# =====
def save_audio(filename, fs, signal):
    signal = signal / np.max(np.abs(signal))
    wavfile.write(filename, fs, (signal * 32767).astype(np.int16))

# =====
# LMS Filter
```



```

# =====
def lms_filter(primary, reference, mu, L):
    N = len(primary)

    w = np.zeros(L)
    y = np.zeros(N)    # estimated noise
    e = np.zeros(N)    # enhanced speech

    ref_padded = np.pad(reference, (L - 1, 0), mode='constant')

    for n in range(N):
        x_vec = ref_padded[n:n + L][::-1]

        y[n] = np.dot(w, x_vec)
        e[n] = primary[n] - y[n]

        w = w + mu * e[n] * x_vec

    return e, y, w

# =====
# NLMS Filter
# =====
def nlms_filter(primary, reference, mu, L, epsilon=1e-6):
    N = len(primary)

    w = np.zeros(L)
    y = np.zeros(N)
    e = np.zeros(N)

    ref_padded = np.pad(reference, (L - 1, 0), mode='constant')

    for n in range(N):
        x_vec = ref_padded[n:n + L][::-1]

        y[n] = np.dot(w, x_vec)
        e[n] = primary[n] - y[n]

```

```

        norm_factor = np.dot(x_vec, x_vec) + epsilon
        w = w + (mu / norm_factor) * e[n] * x_vec

    return e, y, w

# =====
# Evaluation Metrics
# =====
def calculate_snr(clean, test):
    noise = clean - test
    signal_power = np.sum(clean ** 2)
    noise_power = np.sum(noise ** 2)

    return 10 * np.log10(signal_power / (noise_power + 1e-10))

def calculate_mse(clean, test):
    return np.mean((clean - test) ** 2)

# =====
# Plot Results
# =====
def plot_results(clean, noisy, lms_out, nlms_out, fs, true_noise,
                 lms_noise, nlms_noise):
    time = np.arange(len(clean)) / fs

    # Waveform comparison
    plt.figure(figsize=(14, 10))

    plt.subplot(4, 1, 1)
    plt.plot(time, clean)
    plt.title("Clean Speech s[n]")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.grid(True)

```

```

plt.subplot(4, 1, 2)
plt.plot(time, noisy)
plt.title("Noisy Speech x[n]")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(4, 1, 3)
plt.plot(time, lms_out)
plt.title("LMS Enhanced Speech e[n]")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(4, 1, 4)
plt.plot(time, nlms_out)
plt.title("NLMS Enhanced Speech e[n]")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()

# Convergence plot
# Better Convergence Plot
lms_mse = (true_noise - lms_noise) ** 2
nlms_mse = (true_noise - nlms_noise) ** 2

# Moving average smoothing
window_size = 500
window = np.ones(window_size) / window_size

lms_mse_smooth = np.convolve(lms_mse, window, mode='valid')
nlms_mse_smooth = np.convolve(nlms_mse, window, mode='valid')

# Convert sample index to time

```

```

time_axis = np.arange(len(lms_mse_smooth)) / fs

plt.figure(figsize=(12, 6))

plt.plot(time_axis, 10 * np.log10(lms_mse_smooth + 1e-10),
         linewidth=2.2, label="LMS")

plt.plot(time_axis, 10 * np.log10(nlms_mse_smooth + 1e-10),
         linewidth=2.2, label="NLMS")

plt.title("Learning Curve / Convergence Comparison", fontsize
        =15)
plt.xlabel("Time (seconds)", fontsize=12)
plt.ylabel("Smoothed Noise Estimation Error (dB)", fontsize=12)

plt.legend(fontsize=11)
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

# Spectrograms
signals = [
    ("Clean Speech", clean),
    ("Noisy Speech", noisy),
    ("LMS Enhanced Speech", lms_out),
    ("NLMS Enhanced Speech", nlms_out)
]

for title, sig in signals:
    f, t, Sxx = spectrogram(sig, fs)

    plt.figure(figsize=(10, 5))
    plt.pcolormesh(t, f, 10 * np.log10(Sxx + 1e-10), shading='
        gouraud')
    plt.title("Spectrogram: " + title)
    plt.xlabel("Time (s)")
    plt.ylabel("Frequency (Hz)")

```

```

        plt.colorbar(label="Power/Frequency (dB)")
        plt.tight_layout()
        plt.show()

# =====
# Main Simulation
# =====
def run_simulation():
    clean_file = "sp17.wav"
    noisy_file = "sp17_car_sn15.wav"

    fs_clean, clean_speech = load_and_normalize(clean_file)
    fs_noisy, noisy_speech = load_and_normalize(noisy_file)

    if fs_clean != fs_noisy:
        raise ValueError("Sampling frequencies do not match.")

    fs = fs_clean

    min_len = min(len(clean_speech), len(noisy_speech))
    clean_speech = clean_speech[:min_len]
    noisy_speech = noisy_speech[:min_len]

    # Actual noise from dataset
    true_noise = noisy_speech - clean_speech

    # Parameter testing
    mu_values = [0.001, 0.005, 0.01]
    L_values = [32, 64, 128]
    delay_values = [0, 5, 10]

    results = []

    best_lms_snr = -999
    best_nlms_snr = -999

    best_lms_output = None

```

```

best_nlms_output = None
best_lms_noise = None
best_nlms_noise = None

input_snr = calculate_snr(clean_speech, noisy_speech)
input_mse = calculate_mse(clean_speech, noisy_speech)

for mu in mu_values:
    for L in L_values:
        for delay in delay_values:

            reference_noise = np.roll(true_noise, delay)
            reference_noise[:delay] = 0

            lms_out, lms_noise, _ = lms_filter(noisy_speech,
                                                reference_noise, mu, L)
            nlms_out, nlms_noise, _ = nlms_filter(noisy_speech,
                                                    reference_noise, mu, L)

            lms_snr = calculate_snr(clean_speech, lms_out)
            nlms_snr = calculate_snr(clean_speech, nlms_out)

            lms_mse = calculate_mse(clean_speech, lms_out)
            nlms_mse = calculate_mse(clean_speech, nlms_out)

            results.append([
                mu, L, delay,
                input_snr,
                lms_snr,
                lms_snr - input_snr,
                nlms_snr,
                nlms_snr - input_snr,
                input_mse,
                lms_mse,
                nlms_mse
            ])

            if lms_snr > best_lms_snr:

```

```

        best_lms_snr = lms_snr
        best_lms_output = lms_out
        best_lms_noise = lms_noise

    if nlms_snr > best_nlms_snr:
        best_nlms_snr = nlms_snr
        best_nlms_output = nlms_out
        best_nlms_noise = nlms_noise

columns = [
    "mu", "Filter Length L", "Delay",
    "Input SNR dB",
    "LMS Output SNR dB",
    "LMS Improvement dB",
    "NLMS Output SNR dB",
    "NLMS Improvement dB",
    "Input MSE",
    "LMS MSE",
    "NLMS MSE"
]

results_df = pd.DataFrame(results, columns=columns)

print("\n==== Performance Results =====")
print(results_df)

results_df.to_csv("ANC_LMS_NLMS_results.csv", index=False)

print("\nBest LMS SNR:", round(best_lms_snr, 2), "dB")
print("Best NLMS SNR:", round(best_nlms_snr, 2), "dB")

# Save enhanced audio
save_audio("lms_enhanced_speech.wav", fs, best_lms_output)
save_audio("nlms_enhanced_speech.wav", fs, best_nlms_output)

print("\nSaved files:")
print("lms_enhanced_speech.wav")
print("nlms_enhanced_speech.wav")

```

```

print("ANC_LMS_NLMS_results.csv")

# Plot best results
plot_results(
    clean_speech,
    noisy_speech,
    best_lms_output,
    best_nlms_output,
    fs,
    true_noise,
    best_lms_noise,
    best_nlms_noise
)

if __name__ == "__main__":
    run_simulation()

```

Listing 1: Python implementation of LMS and NLMS adaptive noise cancellation